WAC : collecter Windows sans y laisser de trace

Centre d'Excellence Cyberdéfense Aérospatiale (CEC), École de l'air et de l'espace

Une collecte se fait sur une machine qui est aussi une preuve. L'outil qui la prélève doit tout prendre, ne rien y écrire, et pouvoir le démontrer. WAC (Windows Artefacts Collector), logiciel libre écrit en C++, prélève 69 artefacts d'un poste Windows en lecture brute, les convertit en JSON sur le poste d'analyse, scelle et journalise chacune de ses opérations. Ses résultats ont été confrontés, valeur par valeur, à ceux de KAPE et des outils d'Eric Zimmerman.

Ce qui est prélevé

Un seul exécutable autonome, lancé depuis une clé, lit les volumes NTFS en brut : `$MFT` et index des répertoires, sans ouvrir un fichier. Les fichiers verrouillés (ruches, journaux) sont lus comme les autres, et aucune date d'accès ne change. 69 artefacts : ruches système et de chaque profil (SAM en entier, BAM, UserAssist, ShimCache, Amcache, shellbags, USB), `$MFT`, `$UsnJrnl`, `$LogFile`, journaux d'événements avec les ressources qui en donnent le texte, Prefetch, raccourcis, jump lists, SRUM, corbeille, tâches planifiées, WMI, persistance, Defender, navigateurs (historique, téléchargements, sessions et onglets fermés), courrier, clés DPAPI, volumes FAT et exFAT des clés USB. Les exécutables de la machine sont lus et leur signature Authenticode vérifiée en mémoire : un binaire Microsoft authentique n'est consigné que par ses empreintes, tout autre est copié. Face à KAPE (`KapeTriage`), WAC prend tout ce que KAPE prend — hormis 5 `desktop.ini` — et 790 fichiers de plus, en 41 s contre 44.

Une procédure respectée

WAC *collecte* sur la machine examinée ou *convertit* sur le poste d'analyse, jamais les deux. Chaque pièce entre dans une **consigne** scellée : son manifeste donne, pour chacune, MD5, SHA-1 et SHA-256, les huit dates NTFS de l'original (`$STANDARD_INFORMATION` et `$FILE_NAME`), son entrée MFT et la méthode de lecture ; l'empreinte du manifeste le scelle. Après la copie initiale, la consigne n'est plus ni lue ni écrite : la conversion vérifie d'abord le sceau et chaque empreinte, refuse une pièce retouchée, puis travaille sur une copie de travail, où sont rejoués les journaux des ruches et des bases ESE. Chaque opération est inscrite dans le **journal d'investigation** (`investigation.json`) : instant, cible, résultat, trace qu'elle est censée laisser, et durée de chaque étape. Deux scripts livrés revérifient une consigne sans WAC.

Du JSON, pas du CSV

Un fichier par artefact, des valeurs typées (nombres, booléens, listes imbriquées), chaque date au 100 ns en UTC et en heure locale du suspect, avec le décalage en vigueur *à cette date*. Un champ sans valeur n'est pas émis : ni zéro ni date 1601 qui se liraient comme une donnée. Le texte de chaque événement est rendu comme l'affiche Windows. Le résultat se charge tel quel dans un outil d'analyse.

Aucune trace, mesurée

Sur un clone figé de la machine de test, chaque outil est lancé depuis une clé, sous Sysmon, le journal des modifications NTFS borné avant et après. WAC n'écrit **aucun fichier** sur C:, **aucune valeur de registre**, aucun événement dans les journaux de Windows ; il ne lance aucun processus et n'émet rien sur le réseau. Restent l'entrée Amcache et l'entrée ShimCache que Windows crée pour tout programme, et ce qu'une surveillance enregistre de son exécution (Sysmon : le processus, deux lectures brutes de volume). Dans les mêmes conditions, KAPE écrit dans la ruche de l'utilisateur, et ses modules 28 fichiers sur C:. Le script de compilation refuse un `WAC.exe` qui importerait une fonction capable de lancer un processus.

Des tests qui savent échouer

Un cycle autonome, dans une machine virtuelle Windows 11, pose des témoins de valeur connue (fichiers, registre, navigateurs, courrier, un chemin de 261 caractères), collecte, convertit, et passe 494 contrôles. Chacun confronte WAC à une source indépendante : Windows lui-même (`Get-WinEvent`, `Get-FileHash`, registre), `dissect.esedb`, le SQLite et le `lz4` de Python. Une seconde conversion doit donner les mêmes JSON, octet pour octet. Face aux onze outils d'Eric Zimmerman, sur les mêmes pièces, chaque écart a été tranché par une telle source : les valeurs de WAC sont les mêmes, ou plus justes (RECmd ramène des ruches à un état ancien, AmcacheParser invente 3 450 valeurs, EvtxECmd se trompe sur dix). La confrontation a aussi révélé 28 défauts de WAC, tous corrigés avec le contrôle qui les aurait pris.

En chiffres

Artefacts prélevés	69
Collecte sans les exécutables (KAPE : 44 s)	41 s
Conversion complète, consigne vérifiée	80 s
Fichiers écrits sur C : par WAC	0
Contrôles du cycle de test	494
Outils de référence confrontés	12
Défauts de WAC trouvés et corrigés	28

Disponibilité

Version 2.0.0, Windows 10 et 11 (x64), licence GPL 3.0 : `https://github.com/dev-cec/WAC`. Manuels, dossiers de validation, d'innocuité et de comparaison en français et en anglais.